

Advanced Architectures for LLM-Directed Humanoid Kinematics: Waypoint Generation, Chain-of-Thought Reasoning, and State Rollback Mechanisms

Introduction to Embodied Language Models and Kinematic Control

The integration of Large Language Models (LLMs) into humanoid robotics represents a fundamental paradigm shift from rigid, hard-coded state machines to flexible, reasoning-driven orchestration. Traditionally, robotic programming has required extensive domain expertise in mechanical engineering, control theory, and computer science to manually map desired behaviors to joint-specific trajectories and pre-calculated waypoints.¹ These traditional methodologies typically rely on offline programming for predefined trajectories or teach pendant programming for manual path recording.² However, their reliance on static task assumptions limits their applicability in environments characterized by evolving operational demands and open-ended human interactions. The advent of embodied intelligence leverages the vast semantic knowledge embedded in pre-trained foundation models to interpret natural language instructions and dynamically map them to physical actions.³

Despite these profound advances, a critical gap persists between high-level linguistic reasoning and the low-level, continuous control required for complex humanoid kinematics. When deploying an LLM-driven voice agent to guide a humanoid robot's actions, significant challenges arise in ensuring the consistency of waypoint generation, the accurate interpretation of spatial commands, and the reliable management of both conversational and physical state histories. Large Language Models, which are inherently trained on discrete text tokens, often struggle with the continuous spatial and geometric reasoning required to output reliable three-dimensional coordinates or precise joint angles.⁵ Consequently, naive prompting strategies frequently yield hallucinated waypoints, kinematically infeasible trajectories, or highly inconsistent task execution.⁷

Furthermore, humanoid robots operating in dynamic, human-centric environments must be capable of receiving iterative, closed-loop feedback. An instruction such as "that was too slow, try again" necessitates sophisticated, multi-layered mechanisms for conversational memory retrieval, physical state rollback, and dynamic parameter adjustment.⁸ The robot must perfectly recall the previous physical state, revert its actuators safely to that state, understand the

linguistic modifier applied to the previous action, and regenerate the execution command with updated numerical parameters. The analysis contained in this report exhaustively details the architectural frameworks, prompt engineering strategies, and control loop integrations necessary to achieve this highly consistent robotic behavior. By examining state-of-the-art paradigms such as "Code as Policies," "Kinematic-Aware Prompting," and "BrainBody-LLM," the subsequent sections outline the optimal Chain-of-Thought (CoT) structures and state management systems required to reliably translate abstract natural language commands into precise, correctable motor control.

The Geometric Detachment: Analyzing LLM Failures in Direct Waypoint Generation

The core challenge in utilizing an LLM to control physical actuators directly lies in the "geometric detachment" inherent to text-based foundation models. Large Language Models lack an innate understanding of physical grounding, collision constraints, and the continuous nature of three-dimensional space.¹⁰ When prompted to directly output a sequence of 3D coordinates or a matrix of joint angles in radians, these models frequently rely on surficial data patterns rather than transferable, mathematically sound kinematic knowledge.¹¹

In natural language processing, numerical values and spatial coordinates are tokenized in ways that obscure their continuous mathematical relationships. For instance, a spatial coordinate of 0.55 and a coordinate of 0.56 may be represented by entirely different token structures within the model's embedding space. This architectural reality makes it exceedingly difficult for an LLM to perform precise gradient descents, spatial interpolations, or collision-free path planning across a complex trajectory. As a result, the direct regression of joint angles or Cartesian waypoints from an LLM usually results in high error rates, frequent collisions, and unpredictable movements.¹²

The complexity is magnified when dealing with high-Degree-of-Freedom (DoF) humanoid robots. A typical humanoid may feature 43 or more movable axes, requiring synchronized multi-dimensional arrays to coordinate a single fluid motion.¹⁴ Expecting an LLM to autoregressively output the precise mathematical radians for each of these joints simultaneously without violating hardware limits or self-collision boundaries is an improper use of the technology. The literature consistently demonstrates that while LLMs excel at inferring high-level affordances and constraints from free-form language instructions, they fail as direct, low-level numerical controllers.¹⁵ To circumvent this limitation, modern embodied AI frameworks decisively decouple the semantic reasoning capabilities of the LLM from the strict mathematical computation required for spatial routing. Rather than forcing the LLM to act as a numerical calculator, the architecture positions the LLM as a high-level orchestrator that interfaces with deterministic algorithms and low-level control policies.⁵

Architectural Paradigms for Reliable Kinematic Translation

To achieve highly reliable waypoint generation and smooth movement, the robotic control stack must be fundamentally restructured to leverage the LLM's strengths while mitigating its mathematical weaknesses. Several prominent paradigms have emerged to bridge the gap between language models and robotic actuators. The most effective architectures do not prompt the LLM for raw coordinates; instead, they command the LLM to generate executable programming code, manipulate sophisticated 3D value maps, or interface directly with external Inverse Kinematics (IK) solvers.

Code as Policies (CaP)

The "Code as Policies" (CaP) framework represents a highly effective, widely adopted method for ensuring waypoint consistency and enabling complex spatial reasoning. Instead of outputting a sequence of raw coordinates or natural language steps, the LLM is prompted to write executable Python code that parameterizes predefined control primitives and Application Programming Interfaces (APIs).¹⁸ Because LLMs have been trained on billions of lines of source code, they possess a robust, structural understanding of logic, iteration, conditional loops, and variable assignments, making code a far more expressive and reliable medium for spatial reasoning than unstructured text.⁴

Under the CaP paradigm, the LLM generates Language Model Programs (LMPs) that utilize third-party mathematical and geometric libraries such as NumPy and Shapely to handle complex arithmetic calculations.¹⁸ If tasked with moving an arm to form a specific shape or travel a specific distance, the LLM does not arbitrarily guess the coordinates. Instead, it generates a function utilizing precise spatial offsets, such as defining a target location as the current location plus an array vector, allowing the deterministic Python runtime to calculate the exact, continuous waypoint mathematically.¹⁸

The success of the Code as Policies architecture relies heavily on hierarchical code generation. The LLM is prompted to recursively define undefined functions, allowing it to break down highly complex tasks into modular, executable sub-components.¹⁸ This approach significantly reduces hallucinations because the LLM is tightly constrained by the syntax of the programming language and the specific robotic APIs provided in the prompt's context window. Furthermore, by framing the output as code, developers can introduce intermediate verification steps, ensuring that the generated code compiles successfully before it is passed to the physical robot hardware.

VoxPoser and Value Map Generation

An alternative approach to Code as Policies is "VoxPoser," an architecture that aims to synthesize robot trajectories without relying on predefined motion primitives. In this framework, the LLM works in tandem with a Vision-Language Model (VLM) to generate continuous 3D spatial value maps.¹⁵ The LLM analyzes the natural language instruction to logically infer affordances (areas the robot should target) and constraints (areas the robot must avoid). It then writes Python code that interacts with the VLM to project these affordances and constraints into a dense 3D voxel grid representing the robot's immediate physical

workspace.²⁰

Once the value map is synthesized, a traditional, model-based motion planner treats this 3D voxel grid as an objective cost function. The planner synthesizes a dense sequence of 6-DoF end-effector waypoints that maximize the affordance values while simultaneously minimizing constraint costs and avoiding dynamic perturbations.¹⁵ While highly robust for open-ended manipulation and complex obstacle avoidance, VoxPoser requires significant computational overhead to continuously render and optimize 3D voxel grids.²² For environments requiring immediate, low-latency voice interactions and rapid iterative adjustments based on user feedback, a lightweight code-generation approach paired with IK solvers is generally more performant and responsive.

Integrating LLMs with Inverse Kinematics Solvers

For humanoid robots, dictating the exact angle of every individual joint to achieve a coordinated movement is mathematically prohibitive for a language model. To achieve precise, fluid movement, the optimal architecture employs an LLM-IK (Large Language Model - Inverse Kinematics) integration.²³ In this highly specialized structure, the LLM acts exclusively as the semantic and spatial planner, outputting the desired Cartesian target pose of the end-effector (e.g., the position and orientation of the robot's hand in 3D space).

This target Cartesian pose is then passed to an external, highly optimized deterministic IK solver—such as MoveIt, the Kinematics and Dynamics Library (KDL), or TRAC-IK—which mathematically calculates the necessary joint angles via Jacobian matrix solutions to smoothly move the manipulator to the desired target.²⁴ This separation of concerns ensures that the LLM focuses purely on spatial and semantic reasoning, translating the user's verbal command into a geometric goal. Simultaneously, the IK solver handles the complex, non-linear mathematical transformations required to calculate joint velocities, avoid mathematical singularities, and prevent physical self-collisions.²⁷

Paradigm Formulation	Primary Output Modality	Intermediate Processing Layer	Final Physical Execution Mechanism	Optimal Application Profile
Raw Regression	3D Coordinates or Radians	None (Direct Pass-through)	Direct Motor Actuation	Not recommended; suffers from severe hallucination and high collision rates.
Code as Policies	Executable Python LMPs	Python Runtime and Logic Parsing	API Control Primitives	Structured environments requiring specific, logic-driven kinematic paths.
VoxPoser Pipeline	Code defining	VLM + Voxel Value	Optimization	Open-world,

	Affordances	Map Rendering	Motion Planner	obstacle-heavy manipulation requiring deep visual grounding.
LLM-IK Integration	Cartesian End-Effector Poses	Movelt / ROS IK Solvers	Jacobian Matrix Calculations	High-DoF humanoid arms requiring precise, coordinated, and fluid movements.

Structuring Kinematic-Aware Chain-of-Thought (CoT) Prompts

To guarantee that the LLM consistently outputs correct, safe, and parseable commands for the robotic control pipeline, the prompt engineering strategy must enforce a highly structured Chain-of-Thought (CoT) process. CoT prompting significantly improves an LLM's accuracy on complex tasks by forcing the model to explicitly articulate its intermediate reasoning steps before generating a final computational output.²⁹ In the context of humanoid robotics, implementing a "Kinematic-Aware Chain-of-Thought" prompting strategy is absolutely essential for bridging the gap between natural language and physical space.³⁰

The BrainBody-LLM Dual Architecture

The most reliable CoT structures for robotics utilize a multi-agent or dual-LLM framework, prominently conceptualized in the literature as the "BrainBody-LLM" architecture.³¹ This framework divides the substantial cognitive load into two distinct prompts and, in highly optimized systems, two distinct parallel LLM inferences:

1. **The Brain-LLM (High-Level Task Planner):** This module is tasked exclusively with semantic understanding and task decomposition. It analyzes the user's natural language intent, incorporates real-world common-sense knowledge, and breaks the abstract goal down into high-level, logically sequenced steps that can be executed by the agent in the current environment.³²
2. **The Body-LLM (Low-Level Trajectory Executor):** This module receives the decomposed steps from the Brain-LLM and translates them into precise robotic syntax, spatial coordinates, or API control statements.³³ It focuses entirely on the geometry and syntax required to fulfill the sub-task.

By explicitly separating reasoning from control, the system provides a principled alternative to heuristic-driven task planning, allowing for greater fault tolerance and more precise error isolation.

Constructing the Prompt: JSON Schemas and

Kinematic Grounding

To reliably produce good outcomes for spatial commands, the system prompt must explicitly guide the Body-LLM through a meticulously designed reasoning sequence. This involves injecting relevant contextual data about the robot's physical embodiment and utilizing strict output schemas—universally JavaScript Object Notation (JSON)—to ensure downstream robotic systems can parse the data without catastrophic runtime failures.³⁵

An optimal Kinematic-Aware CoT prompt structure for a humanoid robot must follow a strict sequential hierarchy:

Contextual Environment and Kinematic Setup: The prompt must begin by explicitly defining the robot's physical capabilities, its degrees of freedom, the specific nomenclature of its accessible joints (e.g., `left_shoulder_pitch`, `right_elbow_roll`), and the current spatial state of the environment.¹⁸ This grounds the LLM in the physical reality of the specific hardware, preventing it from generating code for non-existent actuators. It is highly recommended to dynamically parse the robot's Universal Robot Description Format (URDF) file and inject a summarized version of the kinematic chain into the prompt's system context.²⁷

Intent Parsing and Sub-Goal Identification: The LLM is instructed to explicitly state the user's goal and identify the specific body parts and control groups required to fulfill the instruction.

Spatial and Geometric Reasoning: This is the core of the CoT step. The LLM must "think aloud" about the geometry of the physical request. It calculates relative proximities, mathematical offsets, and applies relevant physical principles before outputting any code.¹⁷

Structured JSON Output Generation: To prevent the LLM from outputting unstructured conversational text that would crash the execution pipeline, the prompt strictly enforces a JSON schema validation. Free-form prompts force downstream systems to interpret vague responses, whereas structured JSON prompts explicitly define input-output contracts, transforming LLMs into dependable software components.³⁶

JSON Schema Parameter	Data Type	Function and Purpose in the Control Pipeline
<code>reasoning_trace</code>	String	Forces the LLM to articulate geometric logic prior to code generation, enhancing autoregressive accuracy.
<code>target_actuators</code>	Array (Strings)	Identifies the specific robotic joints or IK planning groups required for the upcoming movement.
<code>execution_parameters</code>	Object	Defines necessary modifiers such as desired speed, force limits, or coordinate offsets.
<code>generated_code</code>	String	The fully executable Python Language Model Program (LMP) that interfaces with the

		robot's APIs.
--	--	---------------

By structurally requiring the reasoning_trace to appear first in the JSON object, the LLM is forced to generate the reasoning tokens before it generates the execution tokens. This utilizes the fundamental autoregressive nature of the model, wherein the generation of the code is mathematically conditioned on the preceding logical analysis, dramatically improving the accuracy of the final output.²⁹ Furthermore, enforcing schema validation via libraries such as Pydantic allows the system to automatically check and repair model outputs, ensuring high-fidelity schema adherence.³⁵

Case Study 1: Resolving Complex Spatial Commands ("Form a T")

Translating a highly abstract, geometric command such as "move both arms outwards to form a T" requires the language model to exercise advanced spatial commonsense and deep kinematic awareness. This cannot be achieved by merely mapping keywords to isolated, pre-programmed functions; it requires the system to reason about the robot's global posture and leverage mathematical principles of topological symmetry.

Symmetry-Informed Geometric Reasoning

Humanoid robots inherently possess bilateral symmetry. When generating motions that require coordinated dual-arm manipulation or specific full-body postures, the LLM must be explicitly guided by symmetry-informed prompt architectures.¹⁷

To successfully form a "T" shape, the LLM's reasoning trace—facilitated by the structured CoT schema—must systematically evaluate the geometric requirements of the letter 'T' relative to a humanoid form. A properly engineered prompt will elicit a reasoning trace exhibiting the following logical sequence:

The model determines that a 'T' shape on a humanoid requires the torso to serve as the vertical axis and the arms to serve as the horizontal axis. It deduces that both arms must be extended straight out to the sides, perfectly parallel to the ground. Therefore, the target Cartesian pose for the left end-effector requires a positive offset along the lateral Y-axis (relative to the shoulder origin), with the Z-axis (height) matching the shoulder height exactly. Relying on symmetry principles, the model concludes that the right end-effector requires an equal but negative offset along the lateral Y-axis, with an identical Z-axis height.

Mapping Logic to Motors and IK Solvers

Once the spatial reasoning is firmly established in the context window, the LLM must identify the correct motors and determine the magnitude of movement. Operating within the recommended LLM-IK architecture, the LLM does not calculate the exact, complex radians for the elbow, wrist, and shoulder joints. Instead, the generated code issues a precise Cartesian command to the Inverse Kinematics solver. The generated payload within the generated_code

JSON field would resemble the following logic:

Python

```
# LLM Generated LMP Code for "Form a T"
left_shoulder_origin = get_joint_position('left_shoulder')
right_shoulder_origin = get_joint_position('right_shoulder')

# Define target waypoints based on known humanoid arm length (e.g., 0.6 meters)
# Applying symmetry-informed offsets along the Y-axis
left_target_pose = Pose(x=left_shoulder_origin.x,
                        y=left_shoulder_origin.y + 0.6,
                        z=left_shoulder_origin.z)

right_target_pose = Pose(x=right_shoulder_origin.x,
                        y=right_shoulder_origin.y - 0.6,
                        z=right_shoulder_origin.z)

# Command the IK solver to compute joint angles and move end effectors
execute_ik_trajectory('left_arm_group', left_target_pose, speed=1.0)
execute_ik_trajectory('right_arm_group', right_target_pose, speed=1.0)
```

In scenarios where direct joint angle manipulation is strictly required (bypassing a high-level IK solver), the prompt must contain the absolute kinematic limits and resting states of the robot.³⁹ The LLM would then calculate the specific radians required based on its reasoning trace. For example, moving the arms to a "T" requires adjusting the shoulder roll joints (e.g., moving from 0 radians at the resting side to $\frac{\pi}{2}$ radians, or 90 degrees outward) while explicitly commanding the elbow and wrist pitch/yaw joints to remain locked at 0 radians to maintain a straight geometric line.⁴⁰ The success of this translation relies entirely on the LLM's ability to access the abstracted URDF representation within its context window, ensuring it references the exact motor identifiers assigned by the hardware manufacturer.

Managing Conversational Memory and State History

A critical requirement for advanced, voice-guided robotic agents is their ability to handle follow-up commands and Human-in-the-Loop (HITL) feedback, such as "that was too slow, try again" or "move it a bit higher".¹⁶ Addressing these requests introduces immense architectural complexity, as it requires the LLM to possess perfect conversational memory, the ability to execute physical state rollbacks, and the capacity to dynamically adjust previously executed parameters.

Resolving Token Bloat and the "Lost in Conversation"

Phenomenon

In single-turn interactions, the LLM processes the prompt, generates the code, and stops. However, iterative feedback requires a continuous, stateful "Agent Loop" consisting of Perceive, Reason, Plan, Act, and Observe phases.⁴² If a user says "that was too slow," the pronoun "that" operates as an anaphoric reference. The LLM must reliably retrieve the immediate historical context to understand precisely what action was just performed.

A naive approach to conversational memory simply appends all previous user prompts, visual observations, and generated code blocks to the context window. In multi-agent or iterative robotic workflows, this rapidly leads to "token bloat." As the context window overflows, API latency spikes to unacceptable levels for real-time interaction, and the LLM suffers from a well-documented phenomenon known as "Lost in Conversation" (LiC).⁴³ As the context grows excessively large, the LLM's attention mechanism degrades; it begins to hallucinate, forgets earlier spatial constraints, and fails to maintain intent alignment.

To mitigate this, the architecture must employ a highly optimized, hierarchical session chat history manager.⁴⁴ Effective memory management strategies in robotics dictate the implementation of progressive summarization and code-centric caching.

Memory Tier	Content Stored	Retrieval Mechanism	Purpose in Control Loop
Short-Term (Active)	Last 2-3 user turns + immediately preceding generated Python code.	Direct Context Window	Allows immediate response to modifiers like "try again" or "faster."
Mid-Term (Episodic)	Summaries of completed sub-tasks and spatial landmarks identified.	Semantic Vector Search	Maintains overall task trajectory without bloating the active token limit.
Long-Term (Semantic)	User preferences (e.g., "User is left-handed") and environment maps.	Knowledge Graph / DB Query	Ensures personalization and consistent spatial awareness across sessions.

By maintaining a distinct cache of the *last successfully executed code block*, the agent avoids cluttering the context with conversational pleasantries. When the user gives feedback, only the specific code block relevant to the feedback is re-introduced into the active prompt, alongside the user's critique, ensuring laser-focused parameter adjustment.¹⁸

Case Study 2: Implementing Rollback and Iterative Feedback Loops ("Too slow, try again")

When a user issues a command like "try again," the robot cannot simply execute the new, faster

trajectory from its current, altered physical position. Executing a fresh trajectory from an end-state introduces severe risks of physical collision or self-entanglement. The robot must first reset to its exact kinematic posture prior to the previous command. Implementing an effective rollback mechanism requires a highly synchronized synthesis of software engineering version control and robotic state management.⁴⁷

The Physical State Checkpointing Mechanism

A robust rollback architecture necessitates a dedicated **State History Manager** operating autonomously alongside the LLM pipeline. Before any LLM-generated code or waypoint trajectory is executed by the low-level controller, the robot's complete kinematic state must be checkpointed. This state includes all joint angles, the base position, and the tracked spatial coordinates of relevant environmental objects. This data is saved as a discrete, timestamped snapshot in a time-series database or an in-memory stack.⁹

When the LLM's intent parser detects a user command implying an undo or retry (e.g., "try again," "undo that," "reset"), it maps this semantic intent to a deterministic rollback function rather than attempting to generate inverse kinematics via text.¹⁸ The LLM generates a system command such as `restore_kinematic_state(snapshot_id=-1)`.

The low-level robotic controller intercepts this command and smoothly interpolates the robot's joints from their current physical state back to the exact recorded values of the previous snapshot, effectively rewinding the physical environment.⁹ This explicit, hard-coded rollback mechanism is vital for safety. Relying on an LLM to calculate the mathematical inverse of its previous action—such as attempting to reverse a complex 3D rotation by generating inverse transformation matrices on the fly—is highly prone to compounding errors and physical collisions. Deterministic state caching provides a guaranteed, mathematically safe physical reset.⁵⁰

Dynamic Parameter Adjustment and Speed Modulation

Once the State History Manager has successfully rolled the robot back to its initial position, the LLM must regenerate the action with the requested modifications—in this specific case, executing the motion at a faster speed.

Within the *Code as Policies* paradigm, handling qualitative linguistic modifiers like "faster," "slower," or "higher" relies on the LLM's behavioral commonsense and its ability to prescribe precise numerical values to ambiguous descriptions.¹⁸ To facilitate this, the robotic control APIs provided to the LLM within the system prompt must explicitly expose relevant kinematic parameters.

When the user states "that was too slow," the LLM receives a specialized feedback prompt. This prompt includes the previously executed Python code and the user's critique. The Kinematic-Aware CoT engineering instructs the LLM to identify the specific variable governing the criticized behavior, analyze the required change, and generate the adjusted code.⁵²

For example, the previous execution code stored in the short-term memory cache may have

been:

Python

```
execute_ik_trajectory('both_arms', target_poses, movement_speed=0.5)
```

The LLM, utilizing its structured CoT reasoning trace, evaluates the feedback in the following manner:

1. *Analysis indicates the user feedback classifies the previous execution as "too slow."*
2. *A review of the previous code block reveals a movement_speed parameter set to 0.5.*
3. *To resolve this critique and satisfy the user's intent, the speed parameter must be quantitatively increased.*
4. *Generating updated code replacing movement_speed=0.5 with movement_speed=1.0.*

The newly generated code is then passed to the execution pipeline. This iterative, cyclic process of generating code, executing the physical motion, receiving natural language critique, rolling back the physical state, and regenerating modified code creates a highly adaptive, closed-loop feedback system.³² The programmatic abstraction of "speed" as a discrete parameter allows the LLM to easily manipulate velocity logic without ever needing to recalculate the underlying trajectory gradients or the dense waypoint interpolations. Those complex mathematics remain safely handled by the deterministic, low-level IK controller, ensuring safe and reliable iterative execution.

Synthesizing the Unified Agentic Architecture

To holistically address the complex requirements of consistent waypoint generation, complex spatial interpretation (such as "Form a T"), and reliable iterative feedback with physical rollback ("too slow, try again"), a unified system architecture must be implemented. This architecture synthesizes the strengths of the BrainBody-LLM concept, the Code as Policies paradigm, and deterministic Inverse Kinematics into a cohesive, latency-optimized pipeline.

The Real-Time Execution Flow

1. Multi-Modal Perception and Context Generation: When a voice command is transcribed via speech-to-text, the system immediately polls the robot to gather its current physical state. This includes a structured, JSON-formatted representation of the robot's available control groups, current joint angles, end-effector coordinates, and any relevant environmental bounding boxes.³⁷ This rich context is prepended to the system prompt, providing the LLMs with a real-time snapshot of the physical world.

2. Intent Parsing and Brain-LLM Evaluation: The transcribed command is passed first to the Brain-LLM. The Brain-LLM determines if the command represents a novel action requiring full spatial planning (e.g., "move both arms outwards to form a T") or if it is a feedback and modification command related to the immediately preceding action (e.g., "that was too slow, try again").²² If a modification command is detected, the system queries the Short-Term

Conversational Memory to retrieve the exact Python code that was generated for the previous turn.⁴⁶ It simultaneously triggers the State History Manager to execute a physical rollback, safely returning the robot's joints to the cached state recorded prior to the criticized action.⁹

3. Kinematic-Aware Chain-of-Thought (Body-LLM): The instruction—whether a novel plan or a retry request—is passed to the Body-LLM alongside the Kinematic-Aware CoT prompt.³⁰ The prompt provides few-shot examples of Python API usage, specific instructions on handling symmetry, and guidelines on how to adjust parameters like speed or force.¹⁸ For the "Form a T" command, the CoT explicitly forces the LLM to reason about bilateral symmetry and calculate the necessary Y-axis spatial offsets relative to the shoulders.¹⁷ For the "try again, faster" command, the CoT forces the LLM to identify the speed parameter in the previously retrieved code and increase its numerical value.⁵²

4. Structured Code Generation and Syntax Verification: The Body-LLM outputs a strict JSON object containing the detailed reasoning trace and the generated Python code.³⁶ A lightweight parsing script immediately validates the JSON schema. If the LLM attempts to hallucinate non-existent APIs or introduces Python syntax errors, a localized, automated feedback loop rapidly reprompts the LLM with the error stack trace, forcing a self-correction before any corrupted code reaches the robot's hardware.³²

5. State Checkpointing and Final Execution: Once the code is validated, the current physical state of the robot is cached in the State History Manager to establish a new rollback point.⁴⁷ The generated Python code is then executed by the system runtime. The code calls the external Inverse Kinematics solver, which smoothly translates the LLM-defined Cartesian targets into continuous, collision-free joint trajectories.²⁷ Finally, the low-level controller executes the trajectory at the specific speed dictated by the LLM's parameterized code. This highly decoupled architecture ensures that the Large Language Model is utilized exclusively for what it does best: semantic reasoning, spatial logic, and structural code synthesis. By systematically offloading complex mathematical trajectory generation to dedicated IK solvers, and by offloading physical rollback mechanisms to deterministic state caching, the system entirely mitigates the severe inconsistencies associated with raw LLM waypoint generation. The result is a fluid, responsive, and mathematically safe robotic control pipeline capable of nuanced human interaction.

Works cited

1. LLMs as NAO Robot 3D Motion Planners - CVF Open Access, accessed March 29, 2026, https://openaccess.thecvf.com/content/ICCV2025W/ACVR/papers/Catalini_LLMs_as_NAO_Robot_3D_Motion_Planners_ICCVW_2025_paper.pdf
2. MeCo: Enhancing LLM-Empowered Multi-Robot Collaboration via Similar Task Memoization, accessed March 29, 2026, <https://arxiv.org/html/2601.20577v1>
3. From Text to Motion: Grounding GPT-4 in a Humanoid Robot "Alter3" - arXiv, accessed March 29, 2026, <https://arxiv.org/html/2312.06571v1>
4. LLMs for Robot Task Planning (2025 Guide) | RoboCloud Hub, accessed March 29, 2026, <https://robocloud-dashboard.vercel.app/learn/blog/llm-robot-control>

5. LEVIOSA: Natural Language-Based Uncrewed Aerial Vehicle Trajectory Generation - MDPI, accessed March 29, 2026, <https://www.mdpi.com/2079-9292/13/22/4508>
6. Advancing Spatial Reasoning in Large Language Models: An In-Depth Evaluation and Enhancement Using the StepGame Benchmark, accessed March 29, 2026, <https://ojs.aaai.org/index.php/AAAI/article/view/29811/31406>
7. Planner: A Self-Correcting LLM Framework for Robotic Motion Planning with Temporal Logic, accessed March 29, 2026, <https://arxiv.org/html/2510.16767v1>
8. LLM-Driven Feedback Loops & Their Impact - Emergent Mind, accessed March 29, 2026, <https://www.emergentmind.com/topics/llm-driven-feedback-loops>
9. HandProxy: Expanding the Affordances of Speech Interfaces in Immersive Environments with a Virtual Proxy Hand - arXiv, accessed March 29, 2026, <https://arxiv.org/html/2503.10029v1>
10. EmbodiedVSR: Dynamic Scene Graph-Guided Chain-of-Thought Reasoning for Visual Spatial Tasks - arXiv, accessed March 29, 2026, <https://arxiv.org/html/2503.11089v1>
11. PIVOT-R: Primitive-Driven Waypoint-Aware World Model for Robotic Manipulation - NIPS papers, accessed March 29, 2026, https://proceedings.neurips.cc/paper_files/paper/2024/file/6164b6e5352c139e9ddc1a98c09e4e4a-Paper-Conference.pdf
12. Leveraging Large Language Models for Parametrization and Code Generation of Impedance Controllers in Robotic Manipulation - repositUM, accessed March 29, 2026, <https://repositum.tuwien.at/bitstream/20.500.12708/216441/1/Resch%20Moritz%20-%202025%20-%20Leveraging%20large%20language%20models%20for%20parametrization%20and...pdf>
13. QuantiPhy: A Quantitative Benchmark Evaluating Physical Reasoning Abilities of Vision-Language Models - arXiv, accessed March 29, 2026, <https://arxiv.org/html/2512.19526v1>
14. From text to motion: grounding GPT-4 in a humanoid robot "Alter3" - PMC, accessed March 29, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC12149125/>
15. VoxPoser: Composable 3D Value Maps for Robotic Manipulation with Language Models, accessed March 29, 2026, <https://experts.illinois.edu/en/publications/voxposer-composable-3d-value-maps-for-robotic-manipulation-with-ll/>
16. Composable 3D Value Maps for Robotic Manipulation with Language Models - VoxPoser, accessed March 29, 2026, <https://voxposer.github.io/voxposer.pdf>
17. A Symmetry-Informed Multimodal LLM-Driven Approach to Robotic Object Manipulation: Lowering Entry Barriers in Mechatronics Education - MDPI, accessed March 29, 2026, <https://www.mdpi.com/2073-8994/17/10/1756>
18. Code as Policies: Language Model Programs for Embodied Control, accessed March 29, 2026, <https://code-as-policies.github.io/>
19. [2209.07753] Code as Policies: Language Model Programs for Embodied Control - arXiv, accessed March 29, 2026, <https://arxiv.org/abs/2209.07753>
20. VoxPoser: Composable 3D Value Maps for Robotic Manipulation with Language

- Models, accessed March 29, 2026,
<https://self-supervised.cs.jhu.edu/fa2024/files/presentations/12-3-Robotics-Gao.pdf>
21. VoxPoser: Zero-Shot 3D Value Maps for Robots | PDF - Scribd, accessed March 29, 2026,
<https://www.scribd.com/document/885728281/VoxPoser-Zero-Shot-Robotic-Manipulation-via-Composable-3D-Value-Maps>
 22. A framework for robotic manipulation tasks based on multiple zero shot models - PMC, accessed March 29, 2026,
<https://pmc.ncbi.nlm.nih.gov/articles/PMC12375720/>
 23. Solving Inverse Kinematics using Large Language Models - MSc Thesis Defense by: Steven Rice | School of Computer Science - University of Windsor, accessed March 29, 2026,
<https://www.uwindsor.ca/science/computerscience/372713/solving-inverse-kinematics-using-large-language-models-msc-thesis-defense-steven-rice>
 24. PFEA: An LLM-based High-Level Natural Language Planning and Feedback Embodied Agent for Human-Centered AI - arXiv.org, accessed March 29, 2026,
<https://arxiv.org/html/2510.24109v1>
 25. moveit2_tutorials/doc/concepts/kinematics.rst at main - GitHub, accessed March 29, 2026,
https://github.com/moveit/moveit2_tutorials/blob/main/doc/concepts/kinematics.rst
 26. LLM-IK: Solving Inverse Kinematics Using Large Language Models - IEEE Xplore, accessed March 29, 2026,
<https://ieeexplore.ieee.org/iel8/7083369/11359420/11373900.pdf>
 27. RoboNeuron: A Modular Framework Linking Foundation Models and ROS for Embodied AI, accessed March 29, 2026, <https://arxiv.org/html/2512.10394v1>
 28. Enabling Waypoint Generation for Collaborative Robots using LLMs and Mixed Reality, accessed March 29, 2026, <https://arxiv.org/html/2403.09308v1>
 29. What is Chain of Thought (CoT) Prompting? | NVIDIA Glossary, accessed March 29, 2026, <https://www.nvidia.com/en-us/glossary/cot-prompting/>
 30. Kinematic-aware Prompting for Generalizable Articulated Object Manipulation with LLMs, accessed March 29, 2026, <https://arxiv.org/html/2311.02847v4>
 31. Grounding LLMs For Robot Task Planning Using Closed-loop State Feedback - arXiv, accessed March 29, 2026, <https://arxiv.org/abs/2402.08546>
 32. Grounding LLMs For Robot Task Planning Using Closed-loop State Feedback - arXiv, accessed March 29, 2026, <https://arxiv.org/html/2402.08546v3>
 33. Grounding LLMs For Robot Task Planning Using Closed-loop State Feedback - arXiv, accessed March 29, 2026, <https://arxiv.org/html/2402.08546v2>
 34. Grounding Large Language Models for Robot Task Planning Using Closed-Loop State Feedback - ResearchGate, accessed March 29, 2026,
https://www.researchgate.net/publication/397481894_Grounding_Large_Language_Models_for_Robot_Task_Planning_Using_Closed-Loop_State_Feedback
 35. JSON prompting for LLMs - IBM Developer, accessed March 29, 2026,
<https://developer.ibm.com/articles/json-prompting-llms/>

36. Structured Prompting with JSON: The Engineering Path to Reliable LLMs | by vishal dudd, accessed March 29, 2026, <https://medium.com/@vishal.dutt.data.architect/structured-prompting-with-json-the-engineering-path-to-reliable-llms-2c0cb1b767cf>
37. Lang2Manip: A Tool for LLM-Based Symbolic-to-Geometric Planning for Manipulation 1 Khalifa University Center for Autonomous Robotic Systems (KUCARS), Khalifa University, United Arab Emirates.2 Institute of Industrial and Control Engineering (IOC), Universitat Politècnica de Catalunya, Spain. * This publication is based upon work supported by the - arXiv, accessed March 29, 2026, <https://arxiv.org/html/2512.17062v1>
38. Prompt, Plan, Perform: LLM-based Humanoid Control via Quantized Imitation Learning, accessed March 29, 2026, <https://arxiv.org/html/2309.11359v3>
39. Prompt a Robot to Walk with Large Language Models, accessed March 29, 2026, https://hybrid-robotics.berkeley.edu/publications/CDC2024_Prompt2Walk.pdf
40. CVPR Poster Spatial-Temporal Graph Diffusion Policy with Kinematic Modeling for Bimanual Robotic Manipulation, accessed March 29, 2026, <https://cvpr.thecvf.com/virtual/2025/poster/33065>
41. HUMAN IN THE LOOP: AN APPROACH TO OPTIMIZE LLM BASED ROBOT TASK PLANNING, accessed March 29, 2026, <https://hammer.purdue.edu/ndownloader/files/53805116>
42. What Is the AI Agent Loop? The Core Architecture Behind Autonomous AI Systems, accessed March 29, 2026, <https://blogs.oracle.com/developers/what-is-the-ai-agent-loop-the-core-architecture-behind-autonomous-ai-systems>
43. Intent Mismatch Causes LLMs to Get Lost in Multi-Turn Conversation - arXiv, accessed March 29, 2026, <https://arxiv.org/html/2602.07338v1>
44. Building a Smarter Chat History Manager for AI Chatbots (Session-Level Memory & Context Retrieval) : r/Rag - Reddit, accessed March 29, 2026, https://www.reddit.com/r/Rag/comments/1o3njiv/building_a_smarter_chat_history_manager_for_ai/
45. The Complete Guide to Managing Conversation History in Multi-Agent AI Systems - Medium, accessed March 29, 2026, https://medium.com/@_Ankit_Malviya/the-complete-guide-to-managing-conversation-history-in-multi-agent-ai-systems-0e0d3cca6423
46. Ca\$^2\$P: Cache-Augmented Code-as-Policies for Open-Domain Embodied Tasks, accessed March 29, 2026, <https://openreview.net/forum?id=bZcOscOMlx>
47. Versioning, Rollback & Lifecycle Management of AI Agents: Treating Intelligence as Deployable Software | by NJ Raman | Medium, accessed March 29, 2026, <https://medium.com/@nraman.n6/versioning-rollback-lifecycle-management-of-ai-agents-treating-intelligence-as-deployable-deac757e4dea>
48. Learning to Learn Faster from Human Feedback with Language Model Predictive Control - Robotics, accessed March 29, 2026, <https://www.roboticsproceedings.org/rss20/p125.pdf>
49. WebRollback: Enhancing Web Agents with Explicit Rollback Mechanisms - ACL Anthology, accessed March 29, 2026,

<https://aclanthology.org/2026.eacl-short.12.pdf>

50. Agent architecture: How AI decision-making drives business impact - Retool Blog, accessed March 29, 2026, <https://retool.com/blog/agent-architecture>
51. Catch and revert AI failures in production (automatically) - LaunchDarkly, accessed March 29, 2026, <https://launchdarkly.com/blog/catch-and-revert-ai-failures-in-production/>
52. Evaluating a Hybrid LLM Q-Learning/DQN Framework for Adaptive Obstacle Avoidance in Embedded Robotics - MDPI, accessed March 29, 2026, <https://www.mdpi.com/2673-2688/6/6/115>
53. Act–Observe–Rewrite: Multimodal Coding Agents as In-Context Policy Learners for Robot Manipulation, accessed March 29, 2026, <https://arxiv.org/html/2603.04466v1>
54. GENERATING ROBOT POLICY CODE FOR HIGH-PRECISION AND CONTACT-RICH MANIPULATION TASKS - OpenReview, accessed March 29, 2026, <https://openreview.net/pdf/03b75d32364a898553b63cfbd72655c48cd248d6.pdf>
55. RePrompt: Planning by Automatic Prompt Engineering for Large Language Models Agents, accessed March 29, 2026, <https://arxiv.org/html/2406.11132v2>
56. Inverse Kinematics – ROS 2 Jazzy MoveIt Task Constructor - YouTube, accessed March 29, 2026, <https://www.youtube.com/watch?v=9oYhOwOKi94>